Figma Sync Scripts — Complete Guide

> **Project:** Pills Planner (`Tratamento por Receita`)
> **Figma file:** Parser
> **Runtime:** Node.js ≥ 18 · TypeScript via `tsx`

Table of Contents

1. Overview
2. Prerequisites
3. Initial Setup
    - 3.1 Install dependencies
    - 3.2 Get a Figma Personal Access Token
    - 3.3 Set the FIGMA_TOKEN environment variable
4. Script Reference
    - 4.1 figma-config.ts — Shared configuration
    - 4.2 sync-tokens.ts — Design tokens → CSS
    - 4.3 sync-text-styles.ts — Text styles → TypeScript
    - 4.4 sync-icons.ts — Icon components → SVG
    - 4.5 figma-sync.ts — Orchestrator
5. Running the Scripts
    - 5.1 First-time run (recommended order)
    - 5.2 Subsequent runs
    - 5.3 Selective sync
6. npm Script Shortcuts
7. Output Files
8. Token Validation & Dry-Run Notes
9. Common Errors & Fixes
10. Workflow Diagram
11. Extending the Scripts

---

1. Overview

The sync scripts pull design data from the Figma REST API and write it into the codebase, keeping design tokens, typography, and icons in sync with the Figma source of truth.

Script	Reads from Figma	Writes to codebase
`sync-tokens.ts`	Local variables (colors, spacing, radius)	`src/index.css` `:root` block
`sync-text-styles.ts`	Text styles	`src/lib/text-styles.ts`

Script	Reads from Figma	Writes to codebase
`sync-icons.ts`	`Icon/*` components (SVG export)	`src/assets/icons/*.svg` + `index.ts`
`figma-sync.ts`	All of the above	All of the above

All scripts are written in TypeScript and executed directly with `tsx` — no compilation step required.

---

2. Prerequisites

Requirement	Version	Check
Node.js	≥ 18.0	`node --version`
npm	≥ 9.0	`npm --version`
Figma account	—	figma.com
Access to the Parser file	—	Open file

---

3. Initial Setup

3.1 Install dependencies

Run this once after cloning the repository:

```
npm install
```

This installs `tsx` (listed under `devDependencies`) along with all other project dependencies.

Verify `tsx` is available:

```
npx tsx --version
```

Expected output: `4.x.x`

3.2 Get a Figma Personal Access Token

1. Log in to figma.com
2. Click your avatar (top-right) → **Settings**
3. Scroll to **Security** → **Personal access tokens**
4. Click **Generate new token**
5. Give it a name (e.g. `pill-parse-plan-sync`)
6. Set the following scopes:
    - ✅ **File content** → Read

- ✅ **Variables** → Read

7. Click **Generate token**
8. **Copy the token immediately** — it is only shown once

> ⚠️ **Security:** Never commit your token to version control. The `.env` file is already listed in `.gitignore`.

3.3 Set the FIGMA_TOKEN environment variable

The token must be available as `FIGMA_TOKEN` in the shell that runs the scripts.

Option A — `.env` file (recommended for local development)

Create a `.env` file at the project root (copy from `.env.example`):

```
cp .env.example .env
```

Edit `.env` and replace the placeholder:

```
FIGMA_TOKEN=figd_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

Unix / macOS — source the file before running scripts:

```
source .env && npm run script:sync
```

Windows (PowerShell) — load the file manually:

```
Get-Content .env | ForEach-Object {
  if ($_ -match '^([^#][^=]*)=(.*)$') {
    [System.Environment]::SetEnvironmentVariable($matches[1].Trim(),
$matches[2].Trim(), 'Process')
  }
}
npm run script:sync
```

Windows (Command Prompt):

```
set FIGMA_TOKEN=figd_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
npm run script:sync
```

Option B — Inline (one-off runs)

Unix / macOS / Git Bash:

```
FIGMA_TOKEN=figd_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx npm run script:sync
```

Windows (PowerShell):

```
$env:FIGMA_TOKEN="figd_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"; npm run script:sync
```

Windows (Command Prompt):

```
set FIGMA_TOKEN=figd_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx && npm run script:sync
```

Option C — Shell profile (persistent, Unix/macOS)

Add to `~/.bashrc`, `~/.zshrc`, or equivalent:

```
export FIGMA_TOKEN=figd_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

Then reload:

```
source ~/.zshrc
```

---

4. Script Reference

4.1 `figma-config.ts` — Shared configuration

Path: `scripts/figma-config.ts`
Purpose: Central configuration module imported by all other scripts. Not run directly.

Exports:

Export	Type	Description
`FIGMA_FILE_KEY`	`string`	Figma file key (`q241m9kJPYYMRd448hCbEd`)
`FIGMA_TOKEN`	`string`	Read from `process.env.FIGMA_TOKEN`
`FIGMA_API_BASE`	`string`	`https://api.figma.com/v1`

Export	Type	Description
`TOKEN_CSS_MAP`	`Record<string, string>`	Maps Figma variable names → CSS custom property names
`TOKEN_COLLECTION_NAME`	`string`	Name of the Figma variable collection (`Design Tokens`)
`LIGHT_MODE_NAME`	`string`	Figma mode name for light theme (`Light`)

Customising the token map:

Open `scripts/figma-config.ts` and edit `TOKEN_CSS_MAP`. The key is the Figma variable name (e.g. `color/primary`) and the value is the CSS custom property (e.g. `--primary`):

```
export const TOKEN_CSS_MAP: Record<string, string> = {
  'color/primary': '--primary',
  'spacing/16':    '--spacing-16',
  // add more mappings here
};
```

4.2 `sync-tokens.ts` — Design tokens → CSS

Path: `scripts/sync-tokens.ts`
Purpose: Fetches all local Figma variables (colors, spacing, border radius) and writes them as CSS custom properties into `src/index.css`.

Figma API endpoint used:

```
GET /v1/files/{fileKey}/variables/local
```

Input: Figma variable collection named `Design Tokens`, mode `Light`
Output: Updates the `:root` block in `src/index.css`

Color conversion: Figma colors are stored as `{ r, g, b, a }` floats (0–1 range). The script converts them to HSL format (`H S% L%`) to match the existing shadcn/ui token convention.

CSS marker strategy:

On the **first run**, the script injects two comment markers into the `:root` block:

```
:root {
  /* FIGMA-TOKENS-START */
  --background: 0 0% 100%;
  --primary: 199 89% 48%;
  /* ... */
```

```
  /* FIGMA-TOKENS-END */
}
```

On **subsequent runs**, only the content between the markers is replaced — all other CSS is left untouched.

Run directly:

```
# Unix/macOS
FIGMA_TOKEN=<pat> npx tsx --tsconfig tsconfig.scripts.json scripts/sync-tokens.ts

# Windows PowerShell
$env:FIGMA_TOKEN="<pat>"; npx tsx --tsconfig tsconfig.scripts.json scripts/sync-tokens.ts
```

Expected output:

```
🔍  Fetching Figma variables…
✅  Updated 18 token(s) in src/index.css
```

4.3 `sync-text-styles.ts` — Text styles → TypeScript

Path: `scripts/sync-text-styles.ts`
Purpose: Fetches all text styles from Figma and generates a typed TypeScript constants file at `src/lib/text-styles.ts`.

Figma API endpoints used:

```
GET /v1/files/{fileKey}/styles
GET /v1/files/{fileKey}/nodes?ids={styleKeys}
```

Input: Figma text styles (e.g. `heading/h1`, `body/medium`, `label/small`)
Output: `src/lib/text-styles.ts`

Generated file structure:

```ts
export interface TextStyle {
  fontFamily: string;
  fontSize: number;
  fontWeight: number;
  lineHeight: number;
  letterSpacing: number;
  tailwind: string; // Tailwind class string
}
```

```
export const textStyles = {
  headingH1: {
    fontFamily: 'Inter',
    fontSize: 36,
    fontWeight: 700,
    lineHeight: 1.2,
    letterSpacing: -0.5,
    tailwind: 'text-4xl font-bold leading-[1.2] tracking-[-0.014em]',
  },
  // ...
} satisfies Record<string, TextStyle>;

export type TextStyleKey = keyof typeof textStyles;
```

Usage in components:

```
import { textStyles } from '@/lib/text-styles';

// Use the Tailwind string directly
<h1 className={textStyles.headingH1.tailwind}>Title</h1>

// Or access individual values
const { fontSize, fontWeight } = textStyles.bodyMedium;
```

Run directly:

```
FIGMA_TOKEN=<pat> npx tsx --tsconfig tsconfig.scripts.json scripts/sync-text-styles.ts
```

Expected output:

```
🔍  Fetching Figma text styles…
   Found 9 text style(s).
✅  Generated 9 text style(s) → src/lib/text-styles.ts
```

> **Note:** If the Figma `/nodes` endpoint cannot be reached (e.g. network issue), the script generates stub entries with default values and logs a warning. The file is still created — run again when connectivity is restored.

4.4 `sync-icons.ts` — Icon components → SVG

Path: `scripts/sync-icons.ts`
Purpose: Exports all Figma components whose name starts with `Icon/` as cleaned SVG files, then generates a TypeScript barrel index.

Figma API endpoints used:

```
GET /v1/files/{fileKey}/components
GET /v1/images/{fileKey}?ids={nodeIds}&format=svg
```

Input: Figma components named `Icon/*` (e.g. `Icon/ChevronRight`, `Icon/Pill`)
Output:

- `src/assets/icons/{kebab-name}.svg` — one file per icon
- `src/assets/icons/index.ts` — barrel with `ICON_NAMES` array and `IconName` union type

SVG cleanup applied:

- Removes fixed `width` and `height` attributes (use CSS for sizing)
- Replaces hardcoded `fill="#..."` and `stroke="#..."` with `currentColor` (enables CSS theming)
- Adds `aria-hidden="true" focusable="false"` for accessibility
- Adds `data-icon="{name}"` for traceability

Generated index:

```
export const ICON_NAMES = ['chevron-right', 'pill', 'calendar', ...] as const;
export type IconName = typeof ICON_NAMES[number];
export function getIconUrl(name: IconName): string {
  return `/src/assets/icons/${name}.svg`;
}
```

Usage in components:

```
import type { IconName } from '@/assets/icons';
import { getIconUrl } from '@/assets/icons';

// As an <img>
<img src={getIconUrl('pill')} alt="" aria-hidden="true" className="w-5 h-5" />

// As inline SVG (import the file directly)
import PillIcon from '@/assets/icons/pill.svg?react';
<PillIcon className="w-5 h-5 text-primary" />
```

Run directly:

```
FIGMA_TOKEN=<pat> npx tsx --tsconfig tsconfig.scripts.json scripts/sync-icons.ts
```

Expected output:

```
🔍  Fetching Figma components…
   Found 12 icon component(s).
📥  Requesting SVG exports from Figma…
   ✓ chevron-right.svg
   ✓ pill.svg
   ✓ calendar.svg
   ...
   ✓ src/assets/icons/index.ts

✅  Exported 12 icon(s) → src/assets/icons/
```

> **Note:** Figma limits SVG export requests to ~100 nodes per call. The script batches automatically if there are more than 100 icons.

4.5 `figma-sync.ts` — Orchestrator

Path: `scripts/figma-sync.ts`
Purpose: Runs all three sync scripts in sequence and prints a summary table.

Run directly:

```
FIGMA_TOKEN=<pat> npx tsx --tsconfig tsconfig.scripts.json scripts/figma-sync.ts
```

CLI flags:

Flag	Description	Example
`--only=<names>`	Run only the listed tasks (comma-separated)	`--only=tokens,icons`
`--skip=<names>`	Skip the listed tasks (comma-separated)	`--skip=icons`

Task names: `tokens`, `text-styles`, `icons`

Example — skip icons:

```
FIGMA_TOKEN=<pat> npx tsx --tsconfig tsconfig.scripts.json scripts/figma-sync.ts --skip=icons
```

Example — tokens only:

```
FIGMA_TOKEN=<pat> npx tsx --tsconfig tsconfig.scripts.json scripts/figma-sync.ts --only=tokens
```

Expected output:

```
──────────────────────────────────────────────────────────
  🔄  Figma Sync — 8/7/2026, 10:15:00 AM
──────────────────────────────────────────────────────────
  File: https://www.figma.com/design/q241m9kJPYYMRd448hCbEd/Parser
  Tasks: tokens, text-styles, icons

──────────────────────────────────────────────────────────
  ▶  tokens  —  Design tokens → src/index.css
──────────────────────────────────────────────────────────
🔍  Fetching Figma variables…
✅  Updated 18 token(s) in src/index.css

──────────────────────────────────────────────────────────
  ▶  text-styles  —  Text styles → src/lib/text-styles.ts
──────────────────────────────────────────────────────────
🔍  Fetching Figma text styles…
✅  Generated 9 text style(s) → src/lib/text-styles.ts

──────────────────────────────────────────────────────────
  ▶  icons  —  Icon components → src/assets/icons/
──────────────────────────────────────────────────────────
🔍  Fetching Figma components…
✅  Exported 12 icon(s) → src/assets/icons/

──────────────────────────────────────────────────────────
  📊  Summary
──────────────────────────────────────────────────────────
  ✅  tokens          Design tokens → src/index.css  (2.1s)
  ✅  text-styles     Text styles → src/lib/text-styles.ts  (1.8s)
  ✅  icons           Icon components → src/assets/icons/  (4.3s)

✅  All sync tasks completed successfully.
```

---

5. Running the Scripts

5.1 First-time run (recommended order)

Run the full sync once to generate all output files:

```
# Step 1 — ensure dependencies are installed
npm install

# Step 2 — set your token (Unix/macOS)
export FIGMA_TOKEN=figd_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

# Step 3 — run all syncs
npm run script:sync
```

After this, commit the generated files:

```
git add src/index.css src/lib/text-styles.ts src/assets/icons/
git commit -m "chore: sync design tokens, text styles, and icons from Figma"
```

5.2 Subsequent runs

Run after any design changes in Figma:

```
# Unix/macOS (token already exported in shell)
npm run script:sync

# Or source .env first
source .env && npm run script:sync
```

The scripts are **idempotent** — running them multiple times produces the same result. Only changed values will differ in the git diff.

5.3 Selective sync

When only specific assets changed in Figma:

```
# Only update tokens
npm run script:sync-tokens

# Only update text styles
npm run script:sync-text-styles

# Only update icons
npm run script:sync-icons

# Orchestrator with filter
FIGMA_TOKEN=<pat> npx tsx --tsconfig tsconfig.scripts.json scripts/figma-sync.ts --only=tokens
```

---

6. npm Script Shortcuts

All scripts are available as npm commands (token must be set in the environment):

```
npm run script:sync              # Run all syncs
npm run script:sync-tokens       # Tokens only
```

```
npm run script:sync-text-styles  # Text styles only
npm run script:sync-icons        # Icons only
```

These are defined in `package.json` under `"scripts"` and internally call:

```
npx tsx --tsconfig tsconfig.scripts.json scripts/<name>.ts
```

---

7. Output Files

Generated file	Script	Committed?	Notes
`src/index.css`	`sync-tokens.ts`	✅ Yes	Modifies existing file between markers
`src/lib/text-styles.ts`	`sync-text-styles.ts`	✅ Yes	Fully regenerated each run
`src/assets/icons/*.svg`	`sync-icons.ts`	✅ Yes	One file per icon
`src/assets/icons/index.ts`	`sync-icons.ts`	✅ Yes	Barrel index, fully regenerated

> **Recommendation:** Commit generated files so the app builds without requiring a Figma token in CI/CD. Only re-run sync when the Figma design changes.

---

8. Token Validation & Dry-Run Notes

Token validation

Every script validates `FIGMA_TOKEN` at startup. If it is missing or empty, the script exits immediately with exit code `1`:

```
❌ FIGMA_TOKEN environment variable is not set.
   Set it before running:
     export FIGMA_TOKEN=<your_personal_access_token>
```

No Figma API calls are made and no files are written.

No dry-run mode

The scripts do not have a built-in `--dry-run` flag. To preview changes without writing files:

1. Run the script normally
2. Use `git diff` to review what changed:

```
git diff src/index.css
git diff src/lib/text-styles.ts
git diff src/assets/icons/
```

3. Revert if needed:

```
git checkout -- src/index.css src/lib/text-styles.ts src/assets/icons/
```

Idempotency

All scripts are safe to run multiple times. If nothing changed in Figma, the output files will be identical (or differ only in the `Generated:` timestamp in `text-styles.ts`).

---

9. Common Errors & Fixes

❌ `FIGMA_TOKEN environment variable is not set.`

Cause: The `FIGMA_TOKEN` variable is not exported in the current shell.
Fix: See Section 3.3.

`Figma API error 403`

Cause: The token is invalid, expired, or lacks the required scopes.
Fix:

1. Go to Figma → Settings → Security → Personal access tokens
2. Delete the old token and generate a new one
3. Ensure scopes **File content (read)** and **Variables (read)** are enabled
4. Update your `.env` file with the new token

`Figma API error 404`

Cause: The file key in `figma-config.ts` is incorrect, or you do not have access to the file.
Fix:

1. Open the Figma file in your browser
2. Copy the key from the URL: `figma.com/design/{KEY}/...`
3. Update `FIGMA_FILE_KEY` in `scripts/figma-config.ts`

`Figma API error 429`

Cause: Rate limit exceeded (too many requests in a short period).
Fix: Wait 60 seconds and try again. Avoid running the full sync more than once per minute.

⚠️ No matching variables found. Check TOKEN_CSS_MAP in figma-config.ts.

Cause: The Figma variable names in the file do not match the keys in `TOKEN_CSS_MAP`.
Fix:

1. Open the Figma file → **Assets panel** → **Local variables**
2. Note the exact variable names (e.g. `color/primary`, `Colors/Primary`)
3. Update `TOKEN_CSS_MAP` in `scripts/figma-config.ts` to match

⚠️ No icon components found (expected names starting with "Icon/").

Cause: The Figma file has no published components named `Icon/*`.
Fix:

1. In Figma, select the icon frames
2. Right-click → **Create component**
3. Name them following the convention: `Icon/ChevronRight`, `Icon/Pill`, etc.
4. Publish the components (if using a library)

Could not find :root block in src/index.css

Cause: The `src/index.css` file does not contain a `:root {` block.
Fix: Add a `:root {}` block to `src/index.css`. The script will inject the token markers on the next run.

Cannot find module './figma-config.js'

Cause: Running a script from the wrong directory, or using `node` instead of `tsx`.
Fix: Always run scripts from the project root using `npx tsx`:

```
# ✅ Correct
npx tsx --tsconfig tsconfig.scripts.json scripts/sync-tokens.ts

# ❌ Wrong — do not use node directly
node scripts/sync-tokens.ts
```

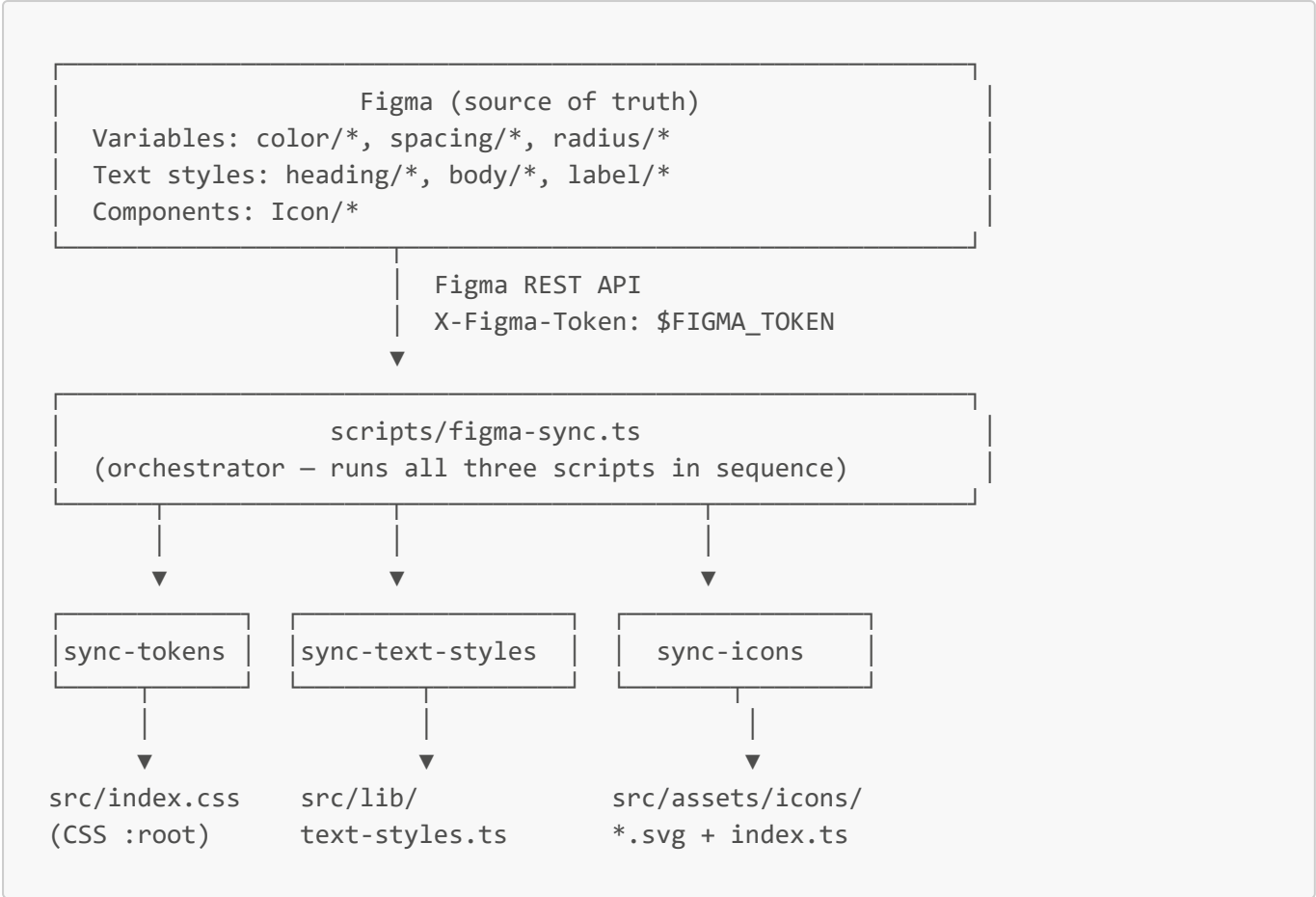
TypeError: fetch is not a function

Cause: Node.js version is below 18 (native `fetch` was added in Node 18).
Fix: Upgrade Node.js to version 18 or later:

```
node --version  # must be v18.0.0 or higher
```

---

10. Workflow Diagram



---

11. Extending the Scripts

Adding a new token mapping

1. Open `scripts/figma-config.ts`
2. Add an entry to `TOKEN_CSS_MAP`:

```
'color/new-token': '--new-token',
```

3. Run `npm run script:sync-tokens`

Adding a new sync script

1. Create `scripts/sync-{name}.ts` following the same pattern as existing scripts
2. Add it to the `TASKS` array in `scripts/figma-sync.ts`:

```
{
  name: 'my-task',
  script: 'scripts/sync-my-task.ts',
  description: 'My task description',
},
```

3. Add an npm shortcut in `package.json`:

```
"script:sync-my-task": "npx tsx --tsconfig tsconfig.scripts.json
scripts/sync-my-task.ts"
```

Changing the Figma file

Update `FIGMA_FILE_KEY` in `scripts/figma-config.ts`:

```
export const FIGMA_FILE_KEY = 'your_new_file_key_here';
```

The file key is the segment between `/design/` and the next `/` in the Figma URL.

Last updated: 2026-08-07 · Auto-generated guide for the Pills Planner project